DBMON-04: Cache and Messaging Monitoring

> **Series:** DBMON — Database Monitoring | **Notebook:** 4 of 7 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Overview

This notebook covers monitoring caches and message brokers with Dynatrace. Caches (Redis, Memcached) and message brokers (Kafka, RabbitMQ) are critical infrastructure components that behave differently from traditional databases. You will learn how to analyze Redis operation performance, Kafka consumer throughput, message broker queue health, and Elasticsearch query patterns using span data.

Table of Contents

1. Cache and Messaging Overview
2. Redis Performance Analysis
3. Kafka Consumer and Producer Monitoring
4. RabbitMQ Queue Monitoring
5. Elasticsearch Query Performance
6. Cross-System Performance Dashboard
7. Summary and Next Steps

Prerequisites

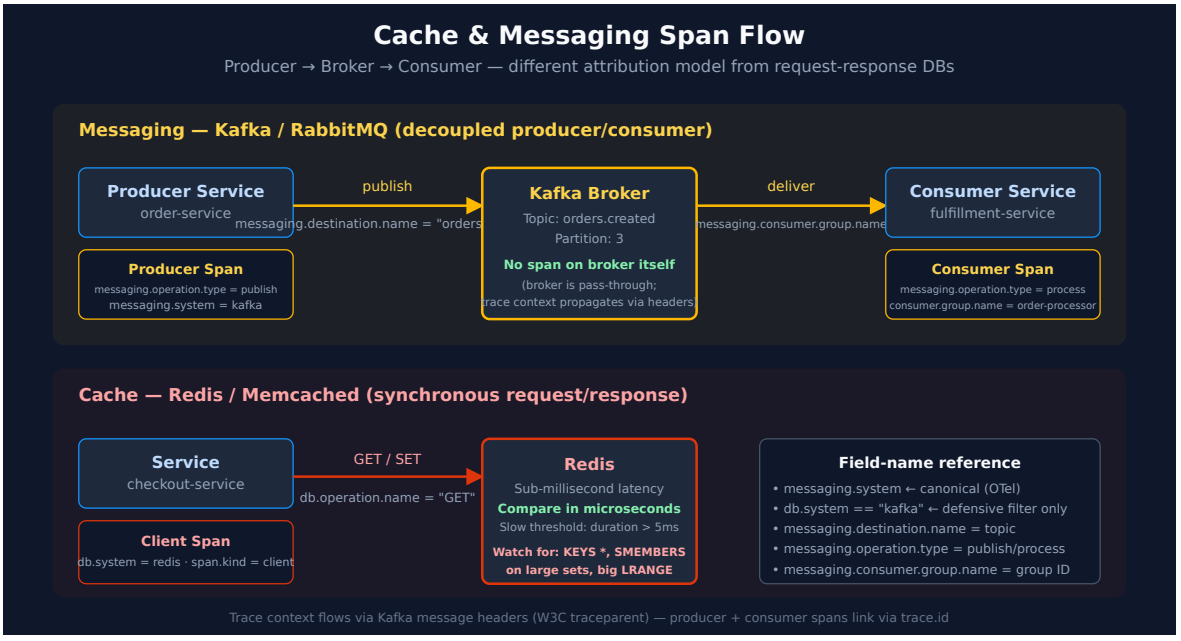
Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail (Managed has no Grail, so these DQL cells do not run there)
OneAgent	Deployed on application hosts using Redis, Kafka, RabbitMQ, or Elasticsearch
Permissions	`storage:spans:read`, `storage:metrics:read`
Data	Application traffic generating cache/messaging calls
Prior Reading	DBMON-01: Database Monitoring Fundamentals

1. Cache and Messaging Overview

Caches and message brokers serve fundamentally different purposes from databases, but Dynatrace monitors them through the same span instrumentation framework.

System	Category	System field	Typical Latency	Key Concern
Redis	In-Memory Cache/Store	`db.system == "redis"`	<1ms	Memory pressure, eviction rate
Memcached	In-Memory Cache	`db.system == "memcached"`	<1ms	Hit ratio, eviction
Kafka	Distributed Streaming	`messaging.system == "kafka"`	1-50ms	Consumer lag, partition balance
RabbitMQ	Message Broker	`messaging.system == "rabbitmq"`	1-10ms	Queue depth, consumer throughput
Elasticsearch	Search Engine	`db.system == "elasticsearch"`	5-500ms	Query latency, indexing throughput

> **Note:** Cache and messaging systems often have sub-millisecond latency. Even small increases in response time can indicate significant issues (memory pressure, network problems, or hot keys).



In community practice the thresholds in this notebook are the values teams converge on as starting points — Dynatrace publishes none of them. Treat each as a first guess to tune against your own baseline, not a recommended setting.

```
// Discover active cache and messaging systems
fetch spans, from:-1h
| filter in(db.system, {"redis", "memcached", "kafka", "rabbitmq",
"elasticsearch", "opensearch"})
| summarize {
    call_count = count(),
    avg_us = avg(duration) / 1us,
    p95_us = percentile(duration, 95) / 1us
}, by:{db.system, server.address}
| sort call_count desc
```

2. Redis Performance Analysis

Redis is the most commonly used in-memory data store. Monitoring Redis involves tracking operation types (GET, SET, DEL, HGET, etc.), latency distribution, and error

rates. Since Redis operations should complete in microseconds, even small latency increases warrant investigation.

Which field carries the command depends on the instrumentation — check `db.operation.name` first, then `code.function` (OneAgent MongoDB) or `span.name` (OTel Redis). The Redis queries below fall back to `span.name` when `db.operation.name` is empty.

```
// Field names corrected 08/12/2026 — pre-1.0 OpenTelemetry database semconv
names had been
// used throughout, and none of them has a row in the semantic dictionary
(older OTel
// instrumentations may still emit db.statement). They fail SILENTLY: a filter
on a
// non-existent field matches nothing and a summarize groups everything under
null, so these cells
// returned empty or single-null-group results without ever erroring.
//   db.operation         -&gt; db.operation.name    (stable; set on 50,379 of
57,295 db spans)
//   db.statement         -&gt; db.query.text        (stable; 33,463)
//   db.mongodb.collection-&gt; db.collection.name   (stable; 6,912)
//   db.name              -&gt; db.namespace         (stable; 57,281)
// Confirm the catalog for your tenant with:
//   fetch dt.semantic_dictionary.fields | filter startsWith(name, "db.") |
fields name, stability
// Redis operation breakdown — which commands are most used?
// OTel Redis instrumentations may omit db.operation.name; the command is also
the span name.
fetch spans, from:-1h
| filter db.system == "redis"
| fieldsAdd operation = coalesce(db.operation.name, span.name)
| summarize {
    call_count = count(),
    avg_us = avg(duration) / 1us,
    p95_us = percentile(duration, 95) / 1us,
    max_us = max(duration) / 1us
}, by:{operation}
| sort call_count desc
```

```
// Redis GET vs SET ratio — understand cache read/write balance
// OTel Redis instrumentations may omit db.operation.name; the command is also
the span name.
fetch spans, from:-1h
| filter db.system == "redis"
| fieldsAdd operation = coalesce(db.operation.name, span.name)
| filter isNotNull(operation)
| fieldsAdd op_type = if(
    in(operation, {"GET", "MGET", "HGET", "HGETALL", "LRANGE", "SMEMBERS",
"ZRANGE"}),
    then:"READ",
    else:if(
      in(operation, {"SET", "MSET", "HSET", "HMSET", "LPUSH", "RPUSH", "SADD",
"ZADD", "DEL", "EXPIRE"}),
      then:"WRITE",
      else:"OTHER"))
| summarize op_count = count(), by:{op_type}
| sort op_count desc
```

```
// Redis latency over time — detect performance degradation
fetch spans, from:-6h
| filter db.system == "redis"
| makeTimeseries {
    avg_us = avg(duration / 1us),
    p95_us = percentile(duration / 1us, 95),
    call_count = count()
  }, interval:5m
```

```
// dt.service.name: dt.entity.service is deprecated in the semantic dictionary
(dt.service.name is stable).
// Redis slow commands — operations exceeding 5ms (abnormal for Redis)
fetch spans, from:-1h
| filter db.system == "redis"
| filter duration &gt; 5ms
// The command is in span.name when db.operation.name is empty (as on the
validation tenant)
| fields start_time, command = coalesce(db.operation.name, span.name),
server.address,
        duration_ms = duration / 1ms, dt.service.name
| sort duration_ms desc
| limit 20
```

> **Tip:** Redis commands like `KEYS *`, `SMEMBERS` on large sets, or `LRANGE` with large ranges are known to cause latency spikes. If you see slow operations, check if the

> command is operating on a large data structure.

3. Kafka Consumer and Producer Monitoring

Apache Kafka monitoring through spans captures both producer (publish) and consumer (process) operations. Dynatrace instruments Kafka client libraries to provide visibility into message throughput, processing latency, and consumer group health.

Kafka Span Attributes

`messaging.system` (`stable` in the semantic dictionary) is the field for Kafka and other message brokers — not `db.system`. The queries below also accept `db.system == "kafka"` defensively, for instrumentations that set it; no source documents which ones do.

Attribute	Description	Example
`messaging.system`	Always `kafka` (canonical OTel field)	`kafka`
`messaging.operation.type`	Dynatrace's dictionary lists `publish`, `receive`, `process` and `peek`; newer OpenTelemetry instrumentations send `send` for producers	`process`
`messaging.destination.name`	Topic name	`orders.created`
`messaging.consumer.group.name`	Consumer group ID	`order-processor-group`
`messaging.destination.partition.id`	Partition number	`3`

> **Dictionary:** `messaging.system`, `messaging.operation.type`, `messaging.destination.name`, `messaging.consumer.group.name`, `messaging.destination.partition.id` (all `stable`), `db.system` (`experimental`),

> read 10/02/2026. No messaging spans existed on the validation tenant over 7 days, so these cells were checked for syntax only.

```
// Kafka message throughput by topic — producer and consumer activity
fetch spans, from:-1h
| filter db.system == "kafka" or messaging.system == "kafka"
| filter isNotNull(messaging.destination.name)
| summarize {
    msg_count = count(),
    avg_ms = avg(duration) / 1ms
}, by:{messaging.destination.name, messaging.operation.type}
| sort msg_count desc
```

```
// Kafka consumer processing time over time by topic
fetch spans, from:-6h
| filter messaging.system == "kafka" and messaging.operation.type == "process"
| filter isNotNull(messaging.destination.name)
| makeTimeseries {
    msg_count = count(),
    avg_process_ms = avg(duration / 1ms)
  }, by:{messaging.destination.name}, interval:5m
```

```
// Kafka consumer group analysis — processing latency by consumer group
fetch spans, from:-1h
| filter messaging.system == "kafka" and messaging.operation.type == "process"
| filter isNotNull(messaging.consumer.group.name)
| summarize {
    msg_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    errors = countIf(span.status_code == "error")
}, by:{messaging.consumer.group.name, messaging.destination.name}
| sort msg_count desc
```

Service Messaging Metrics (`dt.service.messaging.*`)

The span queries above need messaging spans. The latest Dynatrace also records message processing as **metrics on each service**, split by `messaging.destination.name`. These metrics are collected automatically and count messages independently of trace completeness. They replace the Classic Queue entity, and the two models can run side by side while you rebuild views and alerts.

Metric	What it counts
`dt.service.messaging.publish.count`	Messages a service published
`dt.service.messaging.receive.count`	Messages a service received
`dt.service.messaging.process.count`	Messages a service processed
`dt.service.messaging.process.failure_count`	Messages whose processing failed

The query below gives per-topic throughput and failures. Alert on failures as a percentage of `process.count`. The upgrade guide also suggests an alert on the gap between `receive.count` and `process.count` as a processing-lag signal. That signal needs both counts on the consuming service: on the validation tenant (2 h, 11/11/2026) the Kafka topic `orders` reported 38 published and 38 processed messages but **no** received messages. A receive-minus-process lag check there would read negative rather than flag a backlog, so check that `receive.count` is populated for your consumers before you build the lag alert.

> **Sources:** Upgrade queue monitoring to service message processing (DT docs) — *"In the latest Dynatrace, message processing is measured directly on your services through the dt.service.messaging.* metrics."*, *"Both models can run side by side."*, *"An alert on failed messages becomes an alert on dt.service.messaging.process.failure_count, usually expressed as a percentage of dt.service.messaging.process.count."*; re-read 11/11/2026. Query executed 11/11/2026.

```
// Per-topic message throughput and failures from the service messaging
metrics
timeseries {
  published = sum(dt.service.messaging.publish.count),
  received  = sum(dt.service.messaging.receive.count),
  processed = sum(dt.service.messaging.process.count),
  failed    = sum(dt.service.messaging.process.failure_count)
}, from:-2h, by:{messaging.destination.name}, nonempty:true, union:true
| fieldsAdd published_total = arraySum(published), received_total =
arraySum(received),
    processed_total = arraySum(processed), failed_total = arraySum(failed)
| fieldsAdd failure_pct = if(processed_total &gt; 0, round(100.0 *
failed_total / processed_total, decimals: 2))
| fields messaging.destination.name, published_total, received_total,
processed_total, failed_total, failure_pct
| sort processed_total desc
```

4. RabbitMQ Queue Monitoring

RabbitMQ monitoring focuses on message publish/consume rates, queue depth, and consumer processing latency. High queue depth with low consumer throughput indicates backpressure.

```
// RabbitMQ message flow by queue
fetch spans, from:-1h
| filter db.system == "rabbitmq" or messaging.system == "rabbitmq"
| filter isNotNull(messaging.destination.name)
| summarize {
    msg_count = count(),
    avg_ms = avg(duration) / 1ms,
    errors = countIf(span.status_code == "error")
}, by:{messaging.destination.name, messaging.operation.type}
| sort msg_count desc
```

```
// RabbitMQ message throughput trend
fetch spans, from:-6h
| filter messaging.system == "rabbitmq"
| filter isNotNull(messaging.destination.name)
| makeTimeseries msg_count = count(), by:{messaging.operation.type},
interval:5m
```

5. Elasticsearch Query Performance

Elasticsearch is used for full-text search, log aggregation, and analytics. Monitoring focuses on query latency, indexing throughput, and identifying expensive search operations.

```
// Elasticsearch operation breakdown
fetch spans, from:-1h
| filter in(db.system, {"elasticsearch", "opensearch"})
| filter isNotNull(db.operation.name)
| summarize {
    call_count = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms
}, by:{db.system, db.operation.name}
| sort call_count desc
```

```
// Elasticsearch slow queries — search operations exceeding 200ms
fetch spans, from:-1h
| filter in(db.system, {"elasticsearch", "opensearch"})
| filter duration &gt; 200ms
| fields start_time, db.operation.name, db.query.text, server.address,
        duration_ms = duration / 1ms
| sort duration_ms desc
| limit 15
```

6. Cross-System Performance Dashboard

Combining all cache and messaging systems into a single view provides a quick health check across your data infrastructure.

```
// Unified performance summary — all cache and messaging systems
fetch spans, from:-1h
| filter in(db.system, {"redis", "memcached", "kafka", "rabbitmq",
"elasticsearch", "opensearch"})
        or in(messaging.system, {"kafka", "rabbitmq"})
| fieldsAdd system = coalesce(db.system, messaging.system)
| summarize {
    total_ops = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms,
    error_count = countIf(span.status_code == "error")
}, by:{system}
| fieldsAdd error_rate_pct = round((toDouble(error_count) /
toDouble(total_ops)) * 100, decimals:2)
| sort total_ops desc
```

7. Summary and Next Steps

In this notebook you learned:

- Redis operation analysis: command breakdown, read/write ratios, and slow command detection
- Kafka monitoring: producer/consumer throughput, consumer group latency, and topic-level analysis (with canonical `messaging.system`, plus a defensive `db.system == "kafka"` filter)

- RabbitMQ queue monitoring: publish/consume rates and message flow
- Elasticsearch query performance: operation breakdown and slow query detection
- Cross-system performance comparison for cache and messaging infrastructure

Next Steps

- **DBMON-05: Query Analysis** — Deep query analysis including N+1 detection and optimization patterns
- **DBMON-06: Dashboards and Alerting** — Building database monitoring dashboards and alerting rules

Where to Go Deeper

- **OTEL series** (9 notebooks) — OTel-instrumented Kafka/RabbitMQ producers and consumers, semantic-convention reference
- **OPIPE series** (7 notebooks) — OpenPipeline-derived metrics from messaging spans (e.g., per-topic throughput as a metric)
- **OPLOGS series** — When Elasticsearch is used as a log destination upstream of Dynatrace
- **AIOPS series** — Davis anomaly detection on consumer-lag and broker-health metrics

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.